======================================

COVID-19 DATA ANALYSIS PROJECT

======================================

COVID-19 DATA ANALYSIS PROJECT

Introduction

Import Libraries

Load Dataset

Data Cleaning

Exploratory Data Analysis

SQL Queries

Statistical Analysis

Data Visualization

Insights

Conclusion

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
import statistics as st
```

New Section

CLEANING PART

```
df = pd.read_csv("/content/Covid Data.csv")
df.head(5)
df.info()
df.describe()
# df.shape
# df.columns.tolist()

print("\n--- Column Classification ---")
binary_cols = []
non_binary_cols = []

for col in df.columns:
    num_unique = df[col].nunique()
    if num_unique == 2:
        binary_cols.append(col)
    else:
        non_binary_cols.append(col)

print("Binary Columns:", binary_cols)
print("Non-Binary Columns:", non_binary_cols)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1048575 entries, 0 to 1048574
Data columns (total 21 columns):
 #   Column                Non-Null Count    Dtype
---  ------                --------------    -----
 0   USMER                 1048575 non-null  int64
 1   MEDICAL_UNIT          1048575 non-null  int64
 2   SEX                   1048575 non-null  int64
 3   PATIENT_TYPE          1048575 non-null  int64
 4   DATE_DIED             1048575 non-null  object
 5   INTUBED               1048575 non-null  int64
 6   PNEUMONIA             1048575 non-null  int64
 7   AGE                   1048575 non-null  int64
 8   PREGNANT              1048575 non-null  int64
 9   DIABETES              1048575 non-null  int64
 10  COPD                  1048575 non-null  int64
 11  ASTHMA                1048575 non-null  int64
 12  INMSUPR               1048575 non-null  int64
 13  HIPERTENSION          1048575 non-null  int64
 14  OTHER_DISEASE         1048575 non-null  int64
 15  CARDIOVASCULAR        1048575 non-null  int64
 16  OBESITY               1048575 non-null  int64
 17  RENAL_CHRONIC         1048575 non-null  int64
 18  TOBACCO               1048575 non-null  int64
 19  CLASIFFICATION_FINAL  1048575 non-null  int64
 20  ICU                   1048575 non-null  int64
dtypes: int64(20), object(1)
memory usage: 168.0+ MB

--- Column Classification ---
Binary Columns: ['USMER', 'SEX', 'PATIENT_TYPE']
Non-Binary Columns: ['MEDICAL_UNIT', 'DATE_DIED', 'INTUBED', 'PNEUMONIA', 'AG
```

```
# First, convert DATE_DIED to datetime, coercing errors. '9999-99-99' will a
df['DATE_DIED_dt'] = pd.to_datetime(df["DATE_DIED"], errors="coerce")

# Create DIED column: 0 if NaT (survived), 1 if valid date (died)
df['DIED'] = (~df['DATE_DIED_dt'].isna()).astype(int)

# Now, replace the original DATE_DIED column with the datetime version
df['DATE_DIED'] = df['DATE_DIED_dt']
df = df.drop(columns=['DATE_DIED_dt']) # Drop the temporary column

# Replace sentinel values (97, 98, 99) with np.nan for binary columns
binary_cols = ['INTUBED','PNEUMONIA','PREGNANT','DIABETES','COPD',
               'ASTHMA','INMSUPR','HIPERTENSION','OTHER_DISEASE',
               'CARDIOVASCULAR','OBESITY','RENAL_CHRONIC','TOBACCO','ICU']
for col in binary_cols:
  df[col] = df[col].replace({97:np.nan,98:np.nan,99:np.nan})

# Keep only confirmed covid cases (CLASIFFICATION_FINAL values 1, 2, or 3)
df_covid = df[df['CLASIFFICATION_FINAL'].isin([1, 2, 3])].copy()

# Gender mapping
df_covid['GENDER'] =  df_covid['SEX'].map({1:'Male', 2:'Female'})

# Hospitalized status mapping
df_covid['hospitalized'] = df_covid['PATIENT_TYPE'].map({1:'Returned Home',

print(len(df_covid))
print(f"Deaths                : {df_covid['DIED'].sum():,}")
print(f"Survivors             : {(df_covid['DIED']==0).sum():,}")
df_covid.head(25)
```

```
391979
Deaths              : 21,281
Survivors           : 370,698
```

	USMER	MEDICAL_UNIT	SEX	PATIENT_TYPE	DATE_DIED	INTUBED	PNEUMONIA	AG
0	2	1	1	1	2020-03-05	NaN	1.0	6
2	2	1	2	2	2020-09-06	1.0	2.0	5
4	2	1	2	1	NaT	NaN	2.0	6
5	2	1	1	2	NaT	2.0	1.0	4
6	2	1	1	1	NaT	NaN	2.0	6
7	2	1	1	1	NaT	NaN	1.0	6
8	2	1	1	2	NaT	2.0	2.0	3
9	2	1	1	2	NaT	2.0	2.0	2
10	2	1	1	1	NaT	NaN	2.0	3
11	2	1	2	2	NaT	2.0	2.0	2
12	2	1	2	2	NaT	2.0	2.0	3
13	2	1	2	1	NaT	NaN	2.0	5
14	2	1	1	1	NaT	NaN	2.0	4
15	2	1	1	1	NaT	NaN	2.0	2
16	2	1	1	2	NaT	2.0	1.0	8
17	2	1	2	1	NaT	NaN	2.0	6
18	2	1	2	1	NaT	NaN	2.0	5
19	2	1	1	1	NaT	NaN	2.0	6
20	2	1	2	2	NaT	2.0	1.0	5
21	2	1	2	1	NaT	NaN	2.0	3
22	2	1	2	1	NaT	NaN	2.0	4
23	2	1	1	1	NaT	NaN	2.0	2

Missing Value check

```
missing = df_covid.isnull().sum().sort_values(ascending=False)

missing = missing[missing > 0]

print(missing)
```

```
DATE_DIED         370698
ICU               282208
INTUBED           282200
PREGNANT          210872
OTHER_DISEASE       2136
INMSUPR             1449
DIABETES            1440
TOBACCO             1434
CARDIOVASCULAR      1391
HIPERTENSION        1388
OBESITY             1353
RENAL_CHRONIC       1350
COPD                1313
ASTHMA              1309
PNEUMONIA              4
dtype: int64
```

26 rows × 24 columns

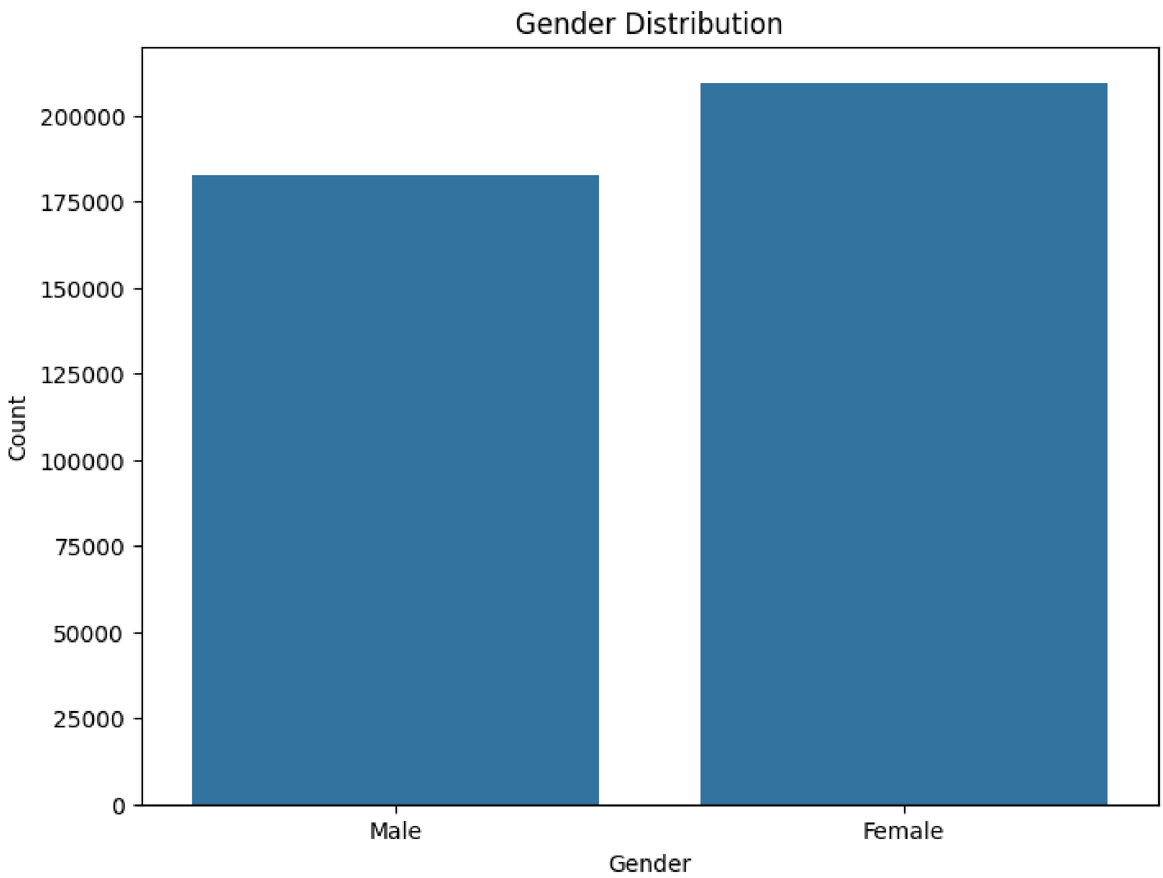
** Death rate **

```
death_rate = df_covid['DIED'].mean() * 100
print("Death Rate:", death_rate)
```

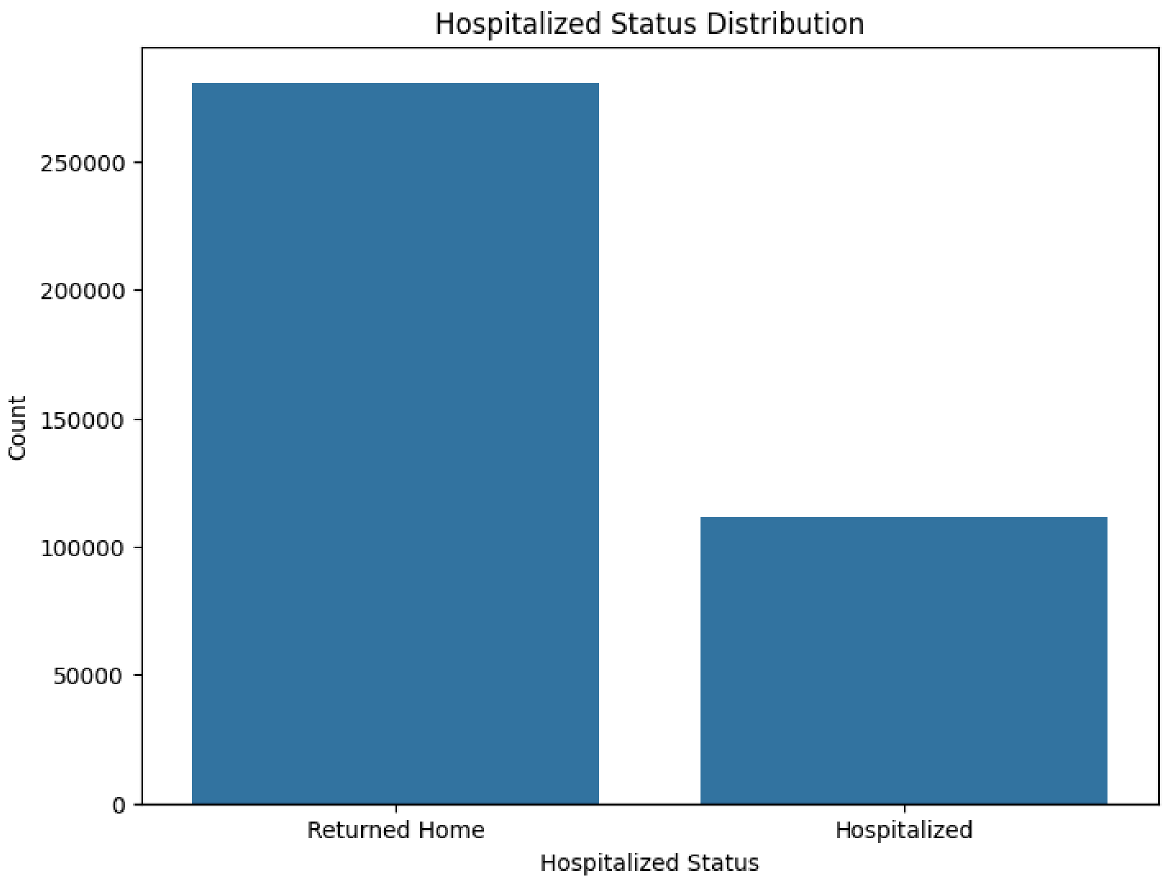
```
Death Rate: 5.429117376185969
```

Gender Distribution

```
plt.figure(figsize=(8, 6))
sns.countplot(data=df_covid, x='GENDER')
plt.title('Gender Distribution')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.show()
```



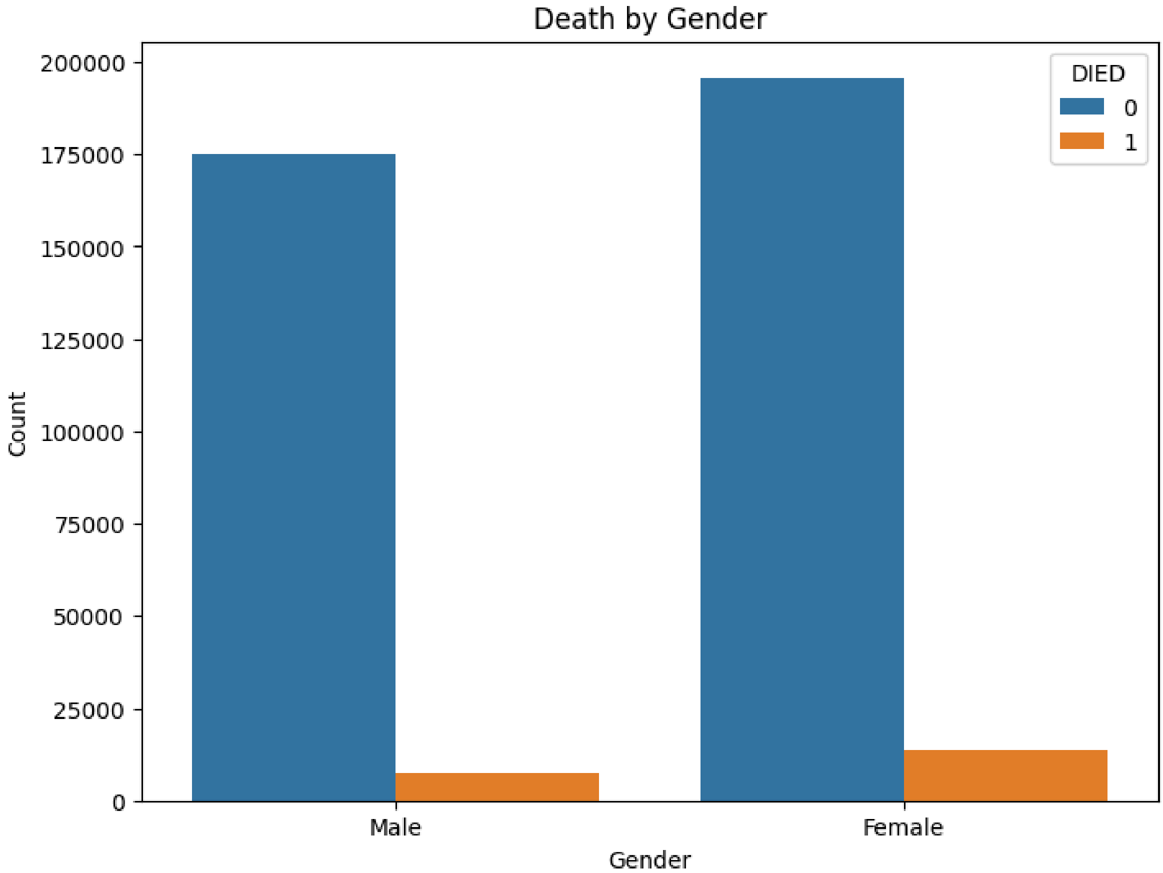

```
plt.figure(figsize=(8, 6))
sns.countplot(data=df_covid, x='hospitalized')
plt.title('Hospitalized Status Distribution')
plt.xlabel('Hospitalized Status')
plt.ylabel('Count')
plt.show()
#
```



** Death by gender **

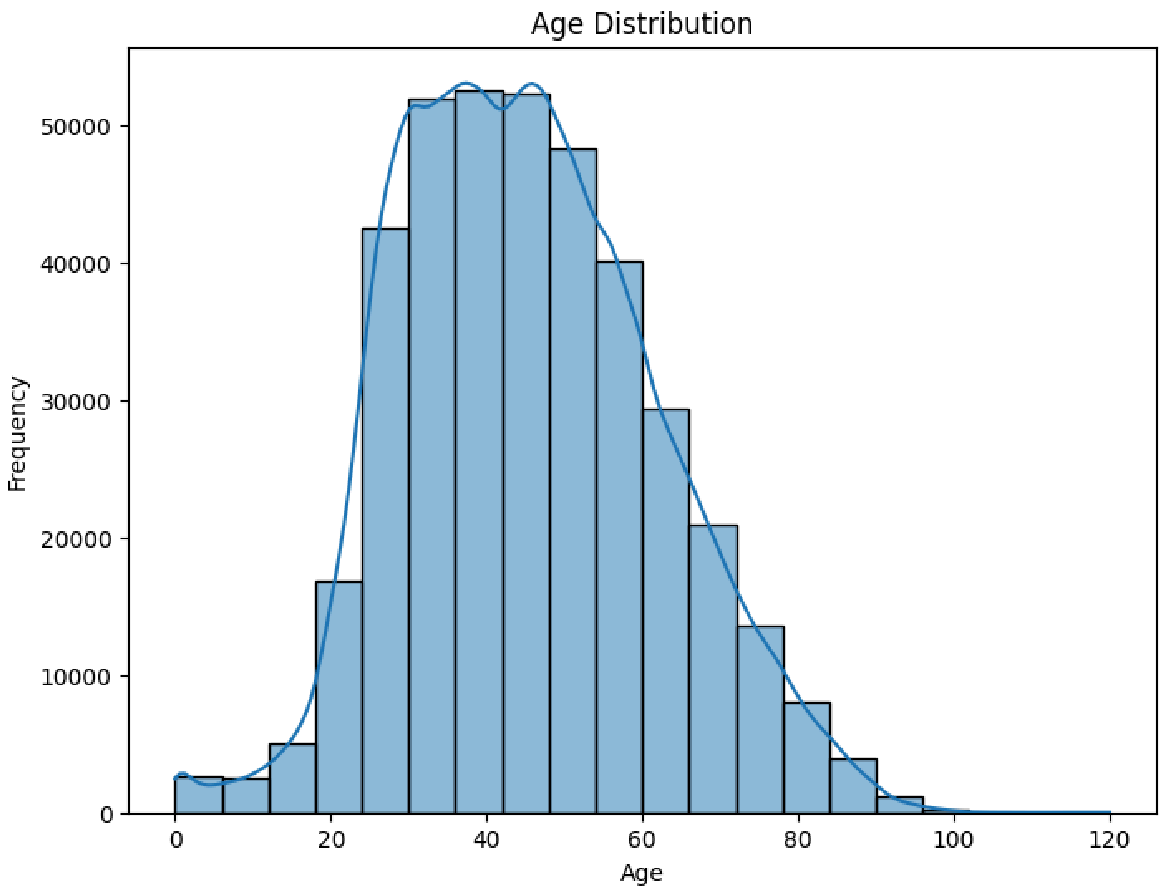
```
plt.figure(figsize=(8 ,6))
sns.countplot(data=df_covid , x= 'GENDER' , hue='DIED')
plt.title('Death by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
```

```
Text(0, 0.5, 'Count')
```



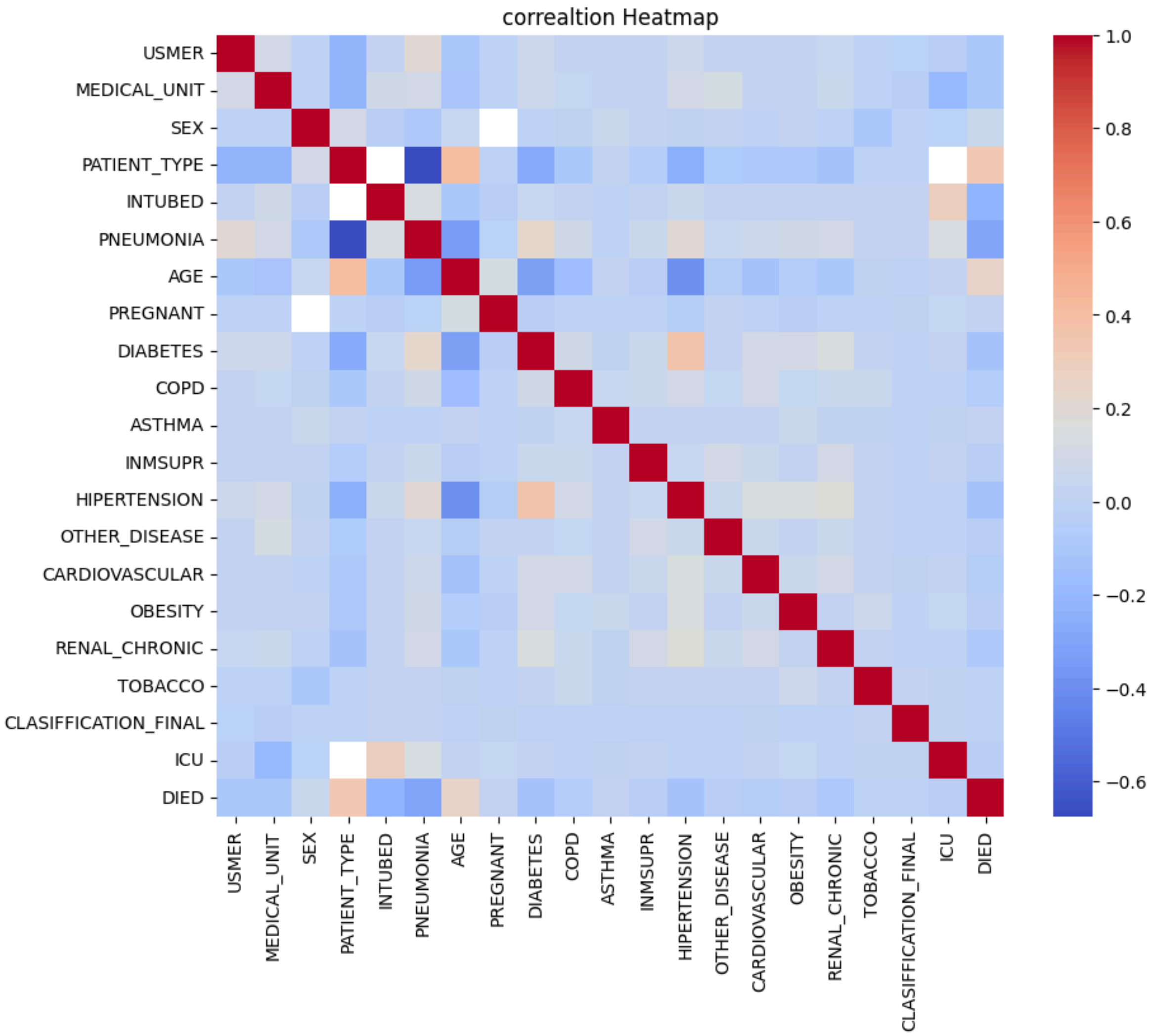
** AGE Disturbution **

```
plt.figure(figsize=(8,6))
sns.histplot(df_covid['AGE'],bins=20, kde=True)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



Corealtion HeatMap

```
plt.figure(figsize=(10,8))
corr =df_covid.corr(numeric_only=True)
sns.heatmap(corr, cmap = 'coolwarm')
plt.title("correaltion Heatmap")
plt.show()
```



SQL CONNECT

```
conn = sqlite3.connect("Memory:")
df_covid.to_sql('covid_data',index = False , con = conn)
print("Connected succesfully:")
```

```
Connected succesfully:
```

SQL QUERIES

Top Age group Most Effected

```
query = """
SELECT
     CASE
        WHEN AGE <20 THEN '1-19'
        WHEN AGE BETWEEN 20 AND 39 THEN '20-39'
        WHEN AGE BETWEEN 40 AND 59 THEN '40-59'
        ELSE '60+'
    END AS AGE_GROUP,

    COUNT(*) AS TOTAL_COUNT,
    SUM(DIED) AS DEATH_COUNT

FROM covid_data
GROUP BY AGE_GROUP
ORDER BY TOTAL_COUNT DESC
"""
result = pd.read_sql_query(query, conn)
print(result)
```

```
  AGE_GROUP  TOTAL_COUNT  DEATH_COUNT
0     40-59        55220         3941
1     20-39        47198          623
2       60+        30935         6700
3      1-19         1979           46
```

DEATH RATE BY GENDER

```
query = """
select
    Gender,
    count(*) as total_count,
    sum(DIED) as death_count
from covid_data
group by Gender
"""
result = pd.read_sql_query(query, conn)
print(result)
```

```
   GENDER  total_count  death_count
0  Female        72448         7175
1    Male        62884         4135
```

Hospitalized vs Returned Home

```
query = """
select
   hospitalized,
   count(*) as total_patients
   from covid_data
   group by hospitalized
"""
result = pd.read_sql_query(query, conn)
print(result)
```

```
    hospitalized  total_patients
0   Hospitalized           53437
1  Returned Home           81895
```

Mortality Rate for diabetes patients

```
query = """
select
    case
    when diabetes = 1 then 'diabtetes'
    else "No Diabetes"
end as diabetes_status,

count(*) as total_cases,
sum(died) as deathes,
round(sum(died)*100.0/count(*),2) as mortality_rate
from covid_data
group by diabetes
"""
result = pd.read_sql_query(query, conn)
print(result)
```

```
  diabetes_status  total_cases  deathes  mortality_rate
0     No Diabetes          165       29           17.58
1       diabtetes        25358     4309           16.99
2     No Diabetes       109809     6972            6.35
```

Top combority count

```
query = """
select
    sum(diabetes = 1) as diabetes,
    sum(HIPERTENSION = 1) as hypertension,
    sum(cardiovascular = 1) as cardiovascular,
    sum(obesity = 1) as obesity,
    sum(renal_chronic = 1) as renal_chronic
    from covid_data;
    """
result = pd.read_sql_query(query, conn)
print(result)
```

```
   diabetes  hypertension  cardiovascular  obesity  renal_chronic
0     25358         32591            3287    25714           4148
```

ICU PATEINTS MORTALITY

```
query="""
select
    case
    when icu = 1 then "ICU"
    when icu = 2 then " NON-ICU"
end as icu_status,
count(*) as total_cases,
sum(died) as deaths,
round(sum(died)*100.0/count(*),2) as mortality_rate
from covid_data
group by icu
"""
result = pd.read_sql_query(query, conn)
print(result)
```

```
  icu_status  total_cases  deaths  mortality_rate
0       None        82339    1190            1.45
1        1.0         1517     297           19.58
2    NON-ICU        51476    9823           19.08
```

Statical Analysis:

Mean median Mode

```
age_data = df_covid['AGE']
mean_age = age_data.mean()
```